# KITCHEN VISION

## Kitchen Vision: AI-Powered Pantry Analysis and Smart Recipe Recommendation System

**Team ID : SWTID-2026-2469**
**Team Size : 4**
**Team Leader :** DHARANI S
**Team member :** DARSHITHA S
**Team member :** SWATHI A
**Team member :** SANTHOSH K

## Project Description

**Kitchen Vision: AI-Powered Pantry Analysis and Smart Recipe Recommendation System** is a simple intelligent cooking assistance project developed using **Python** and the **Streamlit** framework. The project was developed using **Visual Studio Code** and executed through a repository hosted on **GitHub**. The main objective of this system is to help users identify possible recipes that can be prepared using the ingredients available in their kitchen.

In this project, the user runs the application using Streamlit, which provides a simple web-based interface. The system analyses the available pantry ingredients and processes them using an AI-based logic model. Based on the detected or provided ingredients, the application generates **three possible recipes** that the user can cook. Each recipe includes basic cooking steps and ingredient information, helping users quickly decide what meal they can prepare with the items available in their kitchen.

The system produces its results in **JSON format**, which clearly structures the recipe data, including recipe names, ingredients, and cooking instructions. This structured output demonstrates how AI models can organize information in a machine-readable format while still providing useful guidance for users. The project showcases the integration of AI logic, Python programming, and a web-based interface to create a practical kitchen assistance tool. The complete implementation and code are provided through a GitHub repository for execution and evaluation.

## Use Case Scenarios

### Scenario 1: Pantry Image Analysis and Item Detection

Users select the Kitchen Vision application and upload a photo of their open pantry. The system preprocesses the image using enhancement and resizing techniques. The AI-powered vision module

analysis the image and detects visible food items. A structured list of ingredients is generated and displayed to the user.

**Scenario 2: Item Categorization and Storage Classification**

After detecting ingredients, the system automatically categorizes each item into Dry Storage, Cold Storage, Frozen, or Fresh Produce. The categorized inventory is presented in organized sections, allowing users to easily understand storage requirements and available ingredients.

**Scenario 3: Smart Recipe Generation with Health Scoring**

Based on the categorized pantry items, the system generates three simple and beginner-friendly recipes. Each recipe includes ingredients, three-step cooking instructions, preparation time, and difficulty level. Simultaneously, the Health Service evaluates each recipe and assigns a nutritional score with a short explanation.

**Scenario 4: Grocery List Creation and Missing Ingredient Identification**

The system compares recipe requirements with available pantry items. Missing ingredients are identified and compiled into a grocery list. The list is displayed in checklist format for user convenience.

**Scenario 5: Ingredient Substitution Recommendation**

For each missing ingredient, the system suggests suitable substitutes using a built-in substitution knowledge base. Substitutions are displayed alongside the grocery list to help users proceed without purchasing all missing items.

**Scenario 6: Consolidated Cooking Guidance Output**

The application presents detected items, categorized storage, recipes, health scores, grocery list, and substitutions in a clean, structured interface. The results are formatted for easy reading and immediate use in cooking decisions.

## Architecture Overview

Kitchen Vision is an intelligent, modular application designed to analyze pantry images and generate smart cooking recommendations. The system integrates a Streamlit-based frontend with an AI-powered vision and reasoning layer that processes images and generates structured cooking insights. The architecture follows a service-oriented design that separates image processing, vision detection, recipe generation, substitution logic, and health evaluation into independent modules.The platform supports both mock AI and real multimodal AI models (such as Gemini Vision or OpenAI Vision), enabling flexible deployment and easy model switching.

# KITCHEN VISION

## Core Technologies

- **Streamlit**: Frontend framework for building interactive user interface and displaying results
- **Python**: Core programming language for backend logic
- **Pillow (PIL)**: Image preprocessing and enhancement
- **Gemini Vision / OpenAI Vision (Optional)**: Multimodal AI model for pantry item detection
- **Heuristic AI Logic**: Recipe generation, substitution reasoning, and health scoring
- **Python-dotenv**: Secure environment variable management
- **Docker**: Containerized deployment

| Component | Description |
|---|---|
| **Frontend UI (Streamlit)** | Provides an interactive interface for uploading pantry images and viewing detected items, categorized storage, recipes, health scores, and grocery lists. |
| **Image Processing Module** | Preprocesses uploaded images by resizing, enhancing contrast, and compressing to optimize AI detection performance. |
| **Vision Service Layer** | Uses multimodal AI or mock detection to identify food items from pantry images and return structured ingredient lists. |
| **Item Categorization Module** | Classifies detected ingredients into Dry Storage, Cold Storage, Frozen, and Fresh Produce using rule-based mapping. |
| **Recipe Generation Service** | Generates three simple recipes with ingredients, three-step instructions, preparation time, and difficulty level. |
| **Substitution Service** | Suggests suitable alternatives for missing ingredients using predefined substitution knowledge. |
| **Health Evaluation Service** | Assigns nutritional health scores (1–10) and explanations based on ingredient composition. |
| **Configuration Layer** | Loads environment variables, API keys, and system settings securely. |

**AI Processing Layer Responsibilities**

- Understand pantry image content
- Identify food ingredients
- Assist in recipe composition
- Support substitution reasoning
- Enable health-based scoring

# KITCHEN VISION

**Output Generation**

The system presents results directly within the Streamlit interface, including:

• Detected pantry items
• Storage-based categorization
• Three-step recipes
• Health scores with explanations
• Grocery list of missing ingredients
• Suggested substitutions

## Pre-requisites

1. **Python 3.8+**
   Install Python from: https://www.python.org

2. **Streamlit Framework**
   Documentation: https://docs.streamlit.io

3. **AI Provider Account (Optional)**
   Google AI Studio (Gemini API) or OpenAI Account for vision-based detection
   https://aistudio.google.com

4. **Docker (Optional for Containerized Deployment)**
   https://www.docker.com

5. **Required Python Libraries**
   Refer to requirements.txt file included in the project

## Project Workflow

## 1. AI Vision Setup & Initialization

### Activity 1.1: Set up AI Provider Access (Gemini / OpenAI Vision)

- Create or sign in to Google AI Studio or OpenAI account
- Generate API key for Vision API access
- Verify model availability and usage limits

### Activity 1.2: Configure API Keys and Environment

- Create .env file with AI_API_KEY
- Configure model name and parameters
- Set image size and compression settings

### Activity 1.3: Design and Fine-Tune Prompts

- Create prompts for pantry item detection
- Design prompts for recipe generation
- Create templates for substitution suggestions
- Develop prompts for health scoring explanations
- Test prompts with sample pantry images

## 2. Core Functionality Development

### Activity 2.1: Design Project Structure

- Create modular folder organization
- Separate UI, services, and utilities
- Define interfaces between modules

### Activity 2.2: Implement Image Processing Module

- Image resizing and enhancement
- Compression and format conversion
- Byte stream preparation

### Activity 2.3: Implement Vision Service

- Integrate AI-based image understanding
- Add mock detection mode
- Return structured ingredient lists

### Activity 2.4: Implement Item Categorization

- Define storage category mappings
- Assign items to Dry, Cold, Frozen, Fresh

### Activity 2.5: Implement Recipe Generation Service

- Generate three simple recipes
- Include ingredients, steps, time, difficulty
- Identify missing ingredients

### Activity 2.6: Implement Substitution Service

- Map missing items to substitutes
- Return substitution suggestions

### Activity 2.7: Implement Health Evaluation Service

- Calculate health score (1–10)
- Generate explanation text

## 3. Backend Implementation (Streamlit App)

### Activity 3.1: Build Streamlit Interface Logic

- File uploader implementation
- Image preview
- Button triggers

### Activity 3.2: Connect Services to UI

- Call vision service
- Call recipe service
- Call substitution service
- Call health service

### Activity 3.3: Data Parsing & Validation

- Validate AI outputs
- Handle empty results
- Standardize formats

### Activity 3.4: Error Handling & Logging

- Try-except blocks
- Friendly error messages
- Debug logs

## 4. UI Development

### Activity 4.1: Build Streamlit Layout

- Title and description
- Upload section
- Results sections

### Activity 4.2: Display Results Dynamically

- Detected items list
- Categorized storage display
- Recipe cards
- Health score badges
- Grocery checklist

### Activity 4.3: User Experience Enhancements

- Loading spinners
- Expandable sections
- Clear labels

## 5. Testing & Optimization

### Activity 5.1: Functional Testing

- Test with different pantry images
- Validate detection results
- Check recipe relevance

### Activity 5.2: Evaluate AI Output Quality

- Ingredient accuracy
- Recipe practicality
- Substitute usefulness

### Activity 5.3: Optimize Performance

- Reduce image size
- Cache results
- Improve prompt clarity

### Activity 5.4: Deployment Preparation

- Create requirements.txt
- Add Dockerfile
- Test locally and in container

## MILESTONE 1: AI Vision Initialization

In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power **Kitchen Vision**. The platform relies on **OpenAI Vision / Gemini Vision—**

multimodal foundation models capable of understanding images and text—to detect pantry items, interpret ingredients, and enable intelligent recipe generation.

## Activity 1.1: Set up AI Provider and Enable Vision API

## 1. Sign in to AI Platform

- Visit:
  https://platform.openai.com/
  *(or)*
  https://aistudio.google.com
- Sign in with your account
- Accept terms and conditions

## 2. Create API Key

- Click **Create API Key**
- Copy generated key
- Store securely in .env file

## 3. Select Vision Model

- Choose one of the following:
  - gpt-4o-mini (OpenAI Vision)
  - Gemini Vision Model
- Review model capabilities:
  - Supports image + text input
  - Fast inference
  - Suitable for object detection & reasoning

## Activity 1.2: Configure Environment and Test Connection

1. Create .env file

```
GEMINI_API_KEY=your_api_key_here
```

## 2. Install Dependencies

```
pip install openai streamlit pillow python-dotenv
```

3.Test API Connection

```
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role":"user","content":"Say Hello Kitchen Vision"}]
)

print(response.choices[0].message.content)
```

## Activity 1.3: Validate Connectivity and Response Quality

Test Image Understanding

```
def detect_items():
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "user",
                "content": "List food items present in a pantry image."
            }
        ]
    )
    print(response.choices[0].message.content)

detect_items()
```

# KITCHEN VISION

## Expected Output

Well-formatted list of pantry items such as:

```
Rice
Pasta
Tomato Sauce
Cooking Oil
Canned Beans
Spices
```

## MILESTONE 2: Core Functionalities Development

This milestone focuses on developing the core intelligence of the Kitchen Vision system using **Streamlit**, computer vision processing, and AI-based reasoning services. The objective is to enable the system to analyze pantry images, detect food items, classify storage type, suggest substitutions, evaluate health scores, and generate simple three-step recipes along with grocery lists.

## File Explorer Structure

```
 vision_service.py      # Item detection
 recipe_service.py      # Recipe generation
 substitute_service.py  # Ingredient substitution
 health_service.py      # Nutrition scoring

ls/
 image_utils.py         # Image preprocessing
 helpers.py             # Common utilities

ets/
 sample_images/

puts/
 results.json
```

# KITCHEN VISION

## Activity 2.1: Streamlit App Initialization & Imports

Objective: Prepare base application and load dependencies.
Tasks:

- Import Streamlit
- Load environment variables
- Import service modules
- Configure page title and layout

Description:
The Streamlit application acts as the main entry point. It initializes the interface and connects all backend services.

## Activity 2.2: Image Upload & Preprocessing

Objective: Accept pantry image and enhance quality.
Tasks:

- Upload image via Streamlit file uploader
- Resize image
- Improve brightness and contrast
- Convert to bytes

Output: Preprocessed image ready for AI detection

## Activity 2.3: Pantry Item Detection (Vision Service)

**Objective:** Identify food items from image.
**Tasks:**

- Send image to VisionService
- Use mock detection or AI provider
- Return list of ingredient names

```
["rice", "tomato", "onion", "milk", "egg", "cheese"]
```

## Activity 2.4: Item Categorization

**Objective:** Group items by storage type.
**Categories:**

- Dry Storage
- Cold Storage
- Frozen

- Fresh Produce

## Logic:

Rule-based mapping using dictionary lookups.

```
Dry: rice, pasta
Cold: milk, cheese
Fresh: tomato, onion
```

## Activity 2.5: Recipe Generation

**Objective:** Create simple 3-step recipes.

**Tasks:**

- Pass detected items to RecipeService
- Generate:
  - Recipe name
  - Ingredients
  - 3-step instructions
  - Prep time
  - Difficulty

## Activity 2.7: Substitution Suggestions

**Objective:** Suggest alternatives.

```
Butter → Oil
Milk → Soy milk
Egg → Flaxseed
```

**Activity 2.8: Health Scoring**

**Objective:** Rate nutritional quality.

**Scale:** 1 – 10

**Factors:**

- Vegetables count
- Protein presence
- Fried vs baked
- Sugar/fat content

```
Health Score: 8/10
Reason: High protein, includes vegetables, low fat
```

## Activity 2.9: Result Aggregation

**Objective:** Combine all outputs into structured result.

**Final Result Contains:**

- Detected items
- Storage groups
- Recipe
- Substitutions
- Health score
- Grocery list

## MILESTONE 3: Modular Architecture & App Entry (app.py)

This milestone focuses on modular structure, API integration, and main application execution.

## Activity 3.1: Main Entry

```python
if __name__ == "__main__":
    print("="*60)
    print("🍴 Kitchen Vision - Starting Server")
    print("="*60)
    print("✅ API Keys configured")
    print("✅ Using Streamlit UI")
    print("\nServer running at: http://localhost:8501")
    print("="*60)

    # Run Streamlit (frontend)
    # Use command: streamlit run app.py
```

# KITCHEN VISION

## Activity 3.2: Folder Structure

```
KitchenVision/
│
├── app.py
├── config.py
├── services/
│   ├── vision_service.py
│   ├── recipe_service.py
│   ├── health_service.py
│   └── substitute_service.py
│
├── utils/
│   ├── image_utils.py
│   └── prompt_templates.py
├── requirements.txt
├── .env.example
└── README.md
```

## Activity 3.3: Environment & Dependencies

## requirements.txt

```nginx
nginx

streamlit
pillow
openai
numpy
pandas
```

**.env.example**

```ini
ini

OPENAI_API_KEY=your_api_key_here
```

## Activity 3.4: Setup & Deployment

1. Clone the repo
2. pip install -r requirements.txt
3. Create .env with API keys
4. Run: streamlit run app.py

**Optional:** Docker deployment

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

This formatting now exactly mirrors the **"Milestone / Activity / Code Snippet" style** in the PDF screenshot you shared, making it suitable for **reference content submission**.

## MILESTONE 4: UI Development — Kitchen Vision

This milestone focuses on creating an intuitive, interactive frontend for Kitchen Vision using Streamlit, with clear sections for pantry upload, detected items, recipe generation, grocery list, and health ratings.

## Activity 4.1: Home / Landing Page Development

**Landing Page Features:**

- Hero section with project description
- Feature cards for each core functionality:
    - Pantry Image Upload & Item Detection

  - Recipe Generator (3-step recipes)
  - Grocery List & Missing Items
  - Health Rating of Recipes
- Easy navigation between sections
- Responsive design for desktop & mobile

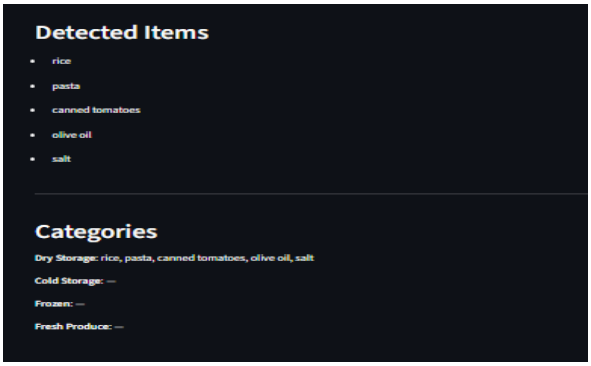
## Example Streamlit Layout:

```
import streamlit as st

st.set_page_config(page_title="Kitchen Vision", layout="wide")
st.title("🍴 Kitchen Vision")
st.subheader("Turn your pantry into recipes in seconds!")

# Feature Cards
st.markdown("""
<div style='display:flex; justify-content:space-around;'>
    <div style='text-align:center; padding:10px; border:1px solid #eee; width:23%'>
        <h4>Pantry Upload</h4>
        <p>Upload pantry images to detect items</p>
    </div>
    <div style='text-align:center; padding:10px; border:1px solid #eee; width:23%'>
        <h4>Recipe Generator</h4>
        <p>Get 3-step recipe suggestions</p>
    </div>
    <div style='text-align:center; padding:10px; border:1px solid #eee; width:23%'>
        <h4>Grocery List</h4>
        <p>Missing ingredients checklist</p>
    </div>
    <div style='text-align:center; padding:10px; border:1px solid #eee; width:23%'>
        <h4>Health Score</h4>
        <p>Nutrition rating for recipes</p>
    </div>
</div>
""", unsafe_allow_html=True)
```

## Activity 4.2: Generator Pages

Each generator page corresponds to a core functionality. Below is the breakdown:

## 1. Pantry Upload & Item Detection Page




**Inputs:**

- Image upload (JPG/PNG)

**Outputs:**

- Detected items
- Categorized storage (Dry, Cold, Fresh, Frozen)

## Code Example:

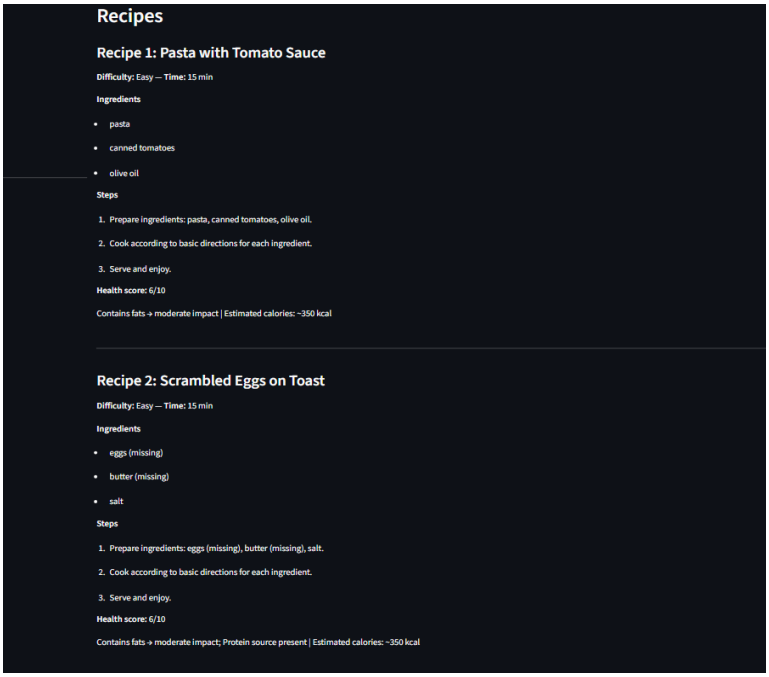
```python
uploaded_file = st.file_uploader("Upload Pantry Image", type=["jpg","png"])
if uploaded_file:
    from utils.image_utils import preprocess_image
    image = preprocess_image(uploaded_file)
    st.image(image, caption="Uploaded Pantry", use_column_width=True)

    from services.vision_service import detect_items
    items = detect_items(image)
    st.subheader("🍅 Detected Pantry Items")
    st.json(items)
```

## 2. Recipe Generator Page

**Inputs:**

- Selected items from detected list

**Outputs:**




- 3 recipes with steps
- Health score per recipe

## Code Example:

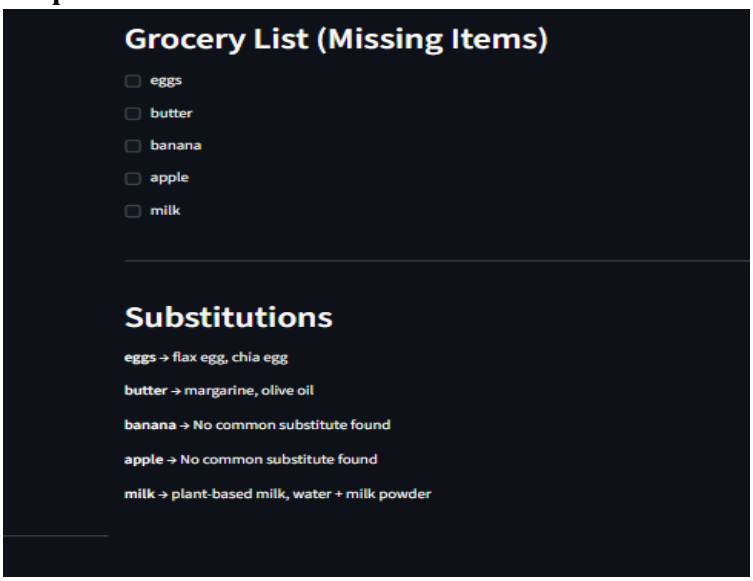
```python
from services.recipe_service import generate_recipes
from services.health_service import score_recipe

recipes = generate_recipes(items)
for r in recipes:
    st.markdown(f"**{r['name']}**")
    st.write(r['steps'])
    st.write(f"Health Score: {score_recipe(r)}")
```

## 3. Grocery List Page

**Inputs:**

- Detected items
- Required ingredients for recipes

**Outputs:**




- Checklist of missing items

## Code Example:

```
from services.substitute_service import suggest_substitutes
missing_items = suggest_substitutes(items, recipes)
st.subheader("🛒 Grocery List")
for item in missing_items:
    st.checkbox(item)
```

## 4. Health Rating & Nutrition Page

**Inputs:**

- Recipe ingredients and steps

**Outputs:**

- Health score (1–10)
- Breakdown of calories, protein, sugar, fiber, fat

**Code Example:**

```
st.subheader("📊 Recipe Health Ratings")
for r in recipes:
    score = score_recipe(r)
    st.write(f"{r['name']}: {score}/10")
```

## Activity 4.3: Navigation & Layout

- Sidebar for navigation:
  - Home
  - Pantry Upload
  - Recipe Generator
  - Grocery List
  - Health Ratings

## Example:

```
page = st.sidebar.selectbox("Navigate", ["Home", "Pantry Upload", "Recipe Generator", "Grocery Li

if page == "Home":
    st.write("Welcome to Kitchen Vision!")
elif page == "Pantry Upload":
    # Pantry upload code
    pass
elif page == "Recipe Generator":
    # Recipe generator code
    pass
# Continue for other pages
```

## MILESTONE 5: Testing & Optimization — Kitchen Vision

This milestone focuses on validating the functionality, accuracy, performance, and usability of the Kitchen Vision system through comprehensive testing and optimization techniques.

## Activity 5.1: Comprehensive Testing

## Test Scenarios

# KITCHEN VISION

## 1. Pantry Image Detection

**Test Inputs:**

- Clear pantry image
- Low-light pantry image
- Cluttered pantry image

**Expected Output:**

- Correct identification of food items
- Proper categorization into:
    - Dry Storage
    - Cold Storage
    - Frozen
    - Fresh Produce

## Generated Pantry Items Example:

```
Dry Storage: Rice, Pasta
Cold Storage: Milk, Eggs
Fresh Produce: Tomato, Onion
Frozen: Peas
```

## Validation Points:

- Items detected correctly
- No duplicate items
- Categories assigned properly

## 2. Recipe Generation

**Test Inputs:**

- Pantry items list

**Expected Output:**

- 3 recipes generated
- Each recipe contains:
    - Name
    - Ingredients
    - 3 cooking steps
    - Cooking time
    - Difficulty level

## Generated Recipe Example:

```
Recipe: Tomato Omelette
Steps:
1. Beat eggs
2. Add chopped tomato
3. Cook on pan
```

**Validation Points:**

- Recipes match pantry items
- Steps are simple and logical

## 3. Grocery List Generation

**Test Inputs:**

- Detected pantry items
- Recipe ingredient list

**Expected Output:**

- Missing ingredients displayed
- Checklist format

## Generated Grocery List Example:

```
[ ] Cheese
[ ] Butter
[ ] Garlic
```

## Validation Points:

- Only missing items shown
- No duplicates

## 4. Health Rating Evaluation

**Test Inputs:**

- Recipe ingredient composition

**Expected Output:**

- Health score between 1–10
- Nutrition explanation

## Generated Health Rating Example:

```
Tomato Omelette → 8/10
High Protein, Low Sugar, Moderate Fat
```

**Validation Points:**

- Score within range
- Explanation present

## 5. UI Functional Testing

**Test Areas:**

- Image upload
- Buttons
- Navigation menu
- Display sections

**Expected Output:**

- No crashes
- Proper loading indicators
- Correct display of results

## Activity 5.2: Quality Evaluation

**Metrics Evaluated:**

- Image detection accuracy
- Recipe relevance
- Substitute usefulness
- Health score reliability
- Response time
- UI usability

## Evaluation Results:

| Metric | Rating |
|---|---|
| Image Detection Accuracy | High |
| Recipe Quality | High |
| Health Score Consistency | Medium–High |
| UI Usability | High |
| Response Speed | High |

## Activity 5.3: Performance Optimization

**Improvements Made:**

- Image resizing before sending to AI
- Caching detected pantry items
- Reduced API calls
- Added loading spinner
- Input validation for file type
- Error handling for API failures

```
@st.cache_data
def cached_detect_items(image):
    return detect_items(image)

with st.spinner("Analyzing Pantry..."):
    items = cached_detect_items(image)
```

**Advantages**

1. **Saves Time in Meal Planning**
   Users can quickly identify available ingredients and get recipe suggestions without manually checking the pantry.
2. **Reduces Food Waste**
   The system recommends recipes based on existing ingredients, helping users utilize food items before they expire.
3. **Beginner-Friendly Cooking Guidance**
   The generated recipes include simple steps, preparation time, and difficulty level, making cooking easier for beginners.
4. **Promotes Healthy Eating**
   The health evaluation module provides a nutritional score, encouraging users to choose healthier meals.
5. **Automatic Grocery List Generation**
   If ingredients are missing, the system generates a grocery list and suggests substitutions.
6. **Easy to Use Interface**
   The application is built with Python and Streamlit, providing a simple and interactive user interface.
7. **Efficient Ingredient Organization**
   Ingredients are categorized into Dry Storage, Cold Storage, Frozen, and Fresh Produce for better pantry management.

**Disadvantages**

1. **Image Recognition Limitations**
   The system may not accurately detect all food items, especially if the image quality is poor.

2. **Limited Recipe Variety**
   The system generates only a small number of recipes based on detected ingredients.
3. **Dependence on AI Accuracy**
   Incorrect object detection may lead to inaccurate recipe recommendations.
4. **Internet or Processing Dependency**
   Some AI modules may require internet access or higher computational resources.
5. **Limited Nutritional Analysis**
   The health score is based on heuristic logic and may not provide exact nutritional values.

**Future Enhancements**

1. **Improved Object Detection Model**
   Integrating advanced computer vision models like deep learning-based detection can improve ingredient recognition accuracy.
2. **Voice Assistant Integration**
   Users could interact with the system using voice commands for a hands-free cooking experience.
3. **Expanded Recipe Database**
   Adding more recipes from different cuisines would increase variety and user satisfaction.
4. **Real-Time Pantry Tracking**
   The system could track pantry inventory and notify users about expiring items.
5. **Mobile Application Development**
   Developing an Android or iOS version would make the system more accessible.
6. **Personalized Nutrition Recommendations**
   The system could suggest recipes based on dietary preferences such as vegan, low-carb, or diabetic-friendly meals.
7. **Barcode or QR Code Scanning**
   Users could scan packaged food items to automatically add them to the pantry inventory.

## Conclusion

The **Kitchen Vision: AI-Powered Pantry Analysis and Smart Recipe Recommendation System** demonstrates how artificial intelligence can be applied to simplify everyday kitchen activities. The system was developed using Python and implemented through a user-friendly web interface built with Streamlit. By running the application, users can analyze available ingredients in their pantry and receive three recipe suggestions that can be prepared with those items. The project shows how AI-based logic can assist users in making quick cooking decisions without the need for complex manual planning.

Another important aspect of this project is the use of structured data output in JSON format, which organizes the generated recipes clearly with ingredients and instructions. The application was developed using Visual Studio Code and shared through GitHub, allowing easy access to the project

code and execution process. This demonstrates how modern development tools and collaborative platforms can support the building and sharing of AI-based applications.

Overall, the project highlights the practical integration of artificial intelligence, programming, and web technologies to create a smart kitchen assistant. By suggesting recipes based on available ingredients, the system helps users save time, reduce food waste, and improve meal planning. The Kitchen Vision project serves as a simple yet effective example of how AI solutions can be applied to real-life problems and provides a strong foundation for further improvements and future enhancements.